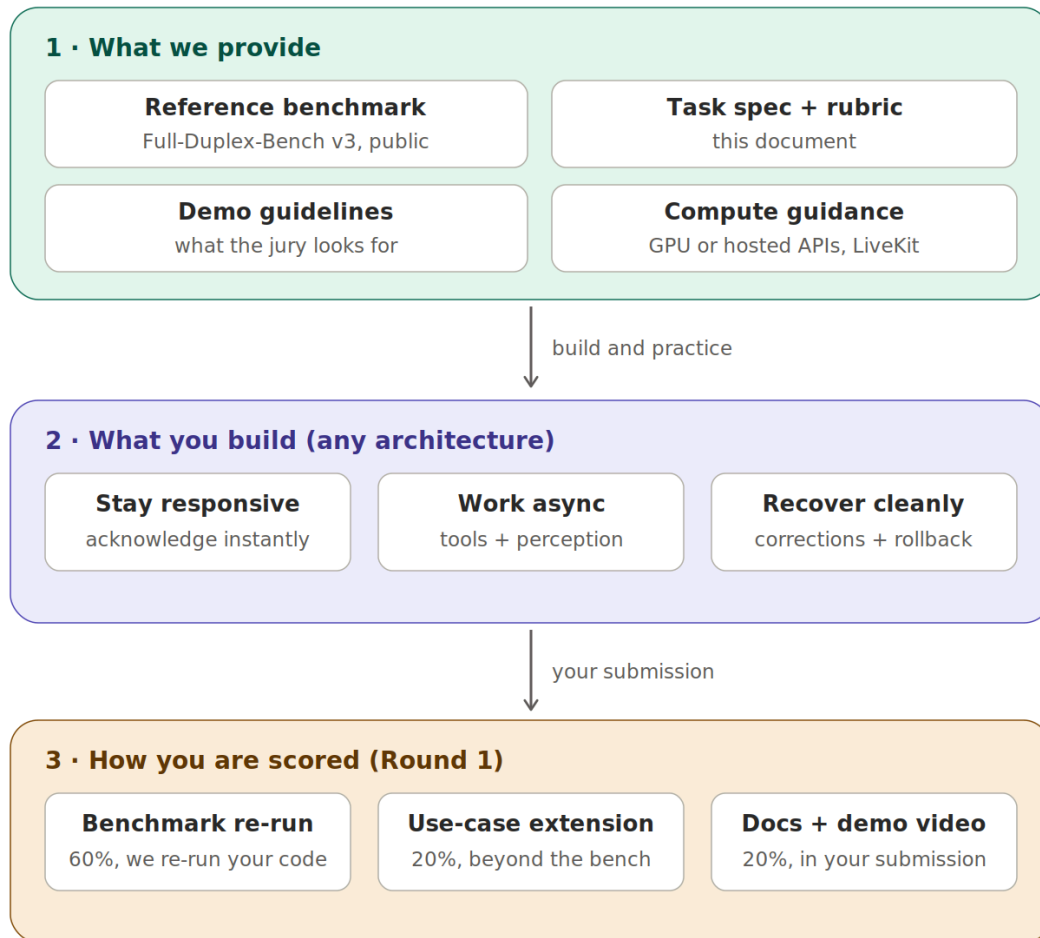


Theme 05: Interruptible Real-Time Agents

Participant guide: what to build, how to evaluate it, what to submit, and how you are scored.



Top teams advance to Round 2: a live jury demo decides the winners and prizes.

At a glance: what we provide, what you build, and how it becomes a score.

1. The challenge

Standard AI assistants operate in half-duplex mode (listen, think, speak). Real people interrupt, hesitate, and correct themselves mid-sentence, and today's assistants fall apart when they do. Think of an in-car assistant dropping a stale route when the destination changes, or a support bot adjusting a booking mid-request without double-booking.

Your task: build a voice-native agent, using any architecture you like, that delivers three capabilities at once:

- **Stay responsive:** meaningful spoken feedback within a few hundred milliseconds; no dead air, no false “done!” claims.
- **Work asynchronously:** tool calls, perception, and reasoning run in the background without ever blocking the conversation.
- **Recover cleanly:** when the user changes their mind mid-utterance, discard stale intent, update tool arguments, and never perform the same state-changing action twice.

2. The reference benchmark

Evaluation is built on **Full-Duplex-Bench v3 (FDB-v3)**, a public benchmark from NTU (with NVIDIA in an advisory role) for exactly this problem: 100 real human recordings (79 unique scenarios, 12 speakers) annotated with five disfluency types (fillers, pauses, hesitations, false starts, self-corrections), paired with chained calls against 12 mock tools across four domains, at three difficulty levels (1, 2, or 3 chained calls). Mock tool outputs are deterministic; scoring runs automatically and uses an LLM judge for semantic argument matching and response quality. It reports tool-selection F1, argument accuracy, strict pass rate, and latency.

- **Repository:** github.com/DanielLin94144/Full-Duplex-Bench (use the v3 directory; paper: arXiv 2604.04847). The benchmark audio is downloaded separately; the link is in the v3 README.
- **Runtime dependency:** agents run inside the LiveKit agents framework, and inference needs a free LiveKit Cloud account. Two agent templates are provided (native realtime models, and a cascaded STT + LLM + TTS pipeline); a custom architecture means writing your own agent script following those patterns. The evaluation scripts themselves do not need LiveKit.
- **Published baselines** for GPT-Realtime, Gemini Live, and a cascaded Whisper-to-LLM pipeline tell you what “good” looks like before you write a line of code.
- **Optional practice:** the same repository's v1/v1.5 (turn-taking and overlap metrics) are useful for tuning interruption behavior, but only v3 is scored.

3. What exactly to do

1. **Build your agent as a LiveKit voice agent.** Any architecture inside that wrapper: realtime speech APIs, a cascaded ASR + LLM + TTS pipeline, open checkpoints, or your own models. Start from the benchmark's two agent templates.
2. **Clone FDB-v3, create a free LiveKit Cloud account, download the benchmark data, and run it against your agent.** Follow the v3 setup instructions in the repository. Your first run will be humbling; that is the point.
3. **Iterate on the failure modes.** Across all published systems, self-correction handling and multi-step tool chains are where scores are lost. That is where you win.
4. **Extend it to one new use case.** Take your agent beyond the benchmark's domains: device troubleshooting with a camera frame, an in-car destination change, a hands-free kitchen assistant, your call. It must run end to end and appear in your video; a sketch on a slide does not count.
5. **Record your demo video, 3 to 5 minutes:** a real interruption being handled on the benchmark, then your extension use case in action. Unedited single takes are preferred over polish.
6. **Submit everything before the deadline** (checklist below). One final submission per team; the last upload counts.

4. What to submit

- **Code repository** with a README covering your architecture (one diagram is plenty), exact setup and run steps, and your extension use case clearly marked.
- **One-command reproduction script** that runs FDB-v3 against your agent end to end (install, configure, evaluate), and a clear declaration of the model provider or custom agent it uses.
- **Your benchmark results and run logs** (scores, seeds, configuration) from your own best run.
- **Demo video, 3 to 5 minutes:** interruption handling on the benchmark, then the extension use case.
- **Slide deck, at most 8 slides:** problem, architecture, benchmark results, what you would do next.

Submit via Google Form. If your agent needs API keys, document which ones and where they go; do not include the keys themselves.

5. How your score is used

Round 1 (all teams) is scored entirely from your submission:

- **Benchmark score, 60%.** We re-run FDB-v3 on your submission using your reproduction script, on a standard machine (single 48 GB GPU, NVIDIA, CUDA 12.x/13.x) or your declared hosted APIs. **Only our re-run counts;** your self-reported numbers guide us but are not scored. If your script does not reproduce, we contact you once; if it still does not run, this portion scores zero, so test the script on a clean machine before submitting.
- **Use-case extension, 20%.** Does your agent do something real beyond the benchmark? Scored on relevance of the use case, whether it runs end to end, and whether the video shows it actually working. One use case done well beats three half-built ones.
- **Documentation, architecture, and video, 20%.** Can we understand and trust your system from the submission alone: a clear README, an honest architecture explanation, and a video that shows real behavior rather than a slideshow.

Official evaluation policy: we run the benchmark with its LLM judge enabled for semantic argument matching and response quality, using a single pinned judge, so every team is judged identically. Supported out of the box: the benchmark's listed realtime providers and its cascaded pipeline; custom LiveKit agents are equally welcome.

Final Round 1 score = $0.6 \times \text{normalized benchmark score} + 0.2 \times \text{extension} + 0.2 \times \text{documentation}$. Ties break on the benchmark's strict pass-rate component.

Round 2 (shortlisted teams only). Top-scoring teams advance to a live jury demo: your agent, interrupted live, plus questions on your design. Winners and prizes are decided in Round 2.

6. Dos and don'ts

- **Do** use public checkpoints and hosted APIs; cite what you use in the README.
- **Do** pin seeds and versions so our re-run matches your logs.
- **Do** test your reproduction script on a machine that is not yours.
- **Do** keep the extension honest: it is scored on working end to end, not on ambition.
- **Don't** hardcode, memorize, or fine-tune on benchmark test items. FDB-v3 is public, so its answers are visible; any submission that pattern-matches test items instead of solving them is disqualified, and we check.
- **Don't** call your own servers at evaluation time; all agent logic lives in the submission.
- **Don't** cache anything across scenarios; each conversation starts fresh.

References:

- <https://artificialanalysis.ai/speech-to-speech?speech-reasoning=speech-reasoning-vs-conversational-dynamics#speech-reasoning-tabs>
- [Full-Duplex-Bench-v3: Benchmarking Tool Use for Full-Duplex Voice Agents Under Real-World Disfluency](#)
- [GitHub - DanielLin94144/Full-Duplex-Bench: A Benchmark for Evaluating Turn-Taking and Overlap Handling in Full-Duplex Spoken Dialogue Models · GitHub](#)